

iSAQB Advanced DSL - Syntax

Michael Sperber

Created: 2026-07-13 Mon 14:53

Lexical Analysis

Read the following program out loud:

```
public class Main {  
    public static void main(String[] args) {  
        // This is a comment  
        System.out.println("Hello World");  
    }  
}
```

Most Programming Languages

- programs divided into **lexemes** ("words")
- spaces and line breaks often mostly irrelevant
- comments are irrelevant
- "Hello World" is a single unit
- `public`, `class` etc. are keywords - built into the language

Lexical Analysis

```
<public> <class> <identifier [Main]> <{>  
<public> <static> <void> <identifier [main]>  
<open paren> <identifier [String]> <open bracket> <closed bracket>  
<identifier [args]> <closed paren>  
<identifier [System]> <dot> <identifier [out]> <dot> <identifier [p  
<open paren> <string [Hello World]> <closed paren> <semicolon>  
<closed brace>  
<closed brace>
```

Lexemes and Tokens

- the letter sequence "Hello World" is a **lexeme**
- its classification as a **string literal** is its **token**
- the text of the string literal Hello World is its **attribute**

(LG 1-2) Lexical Analysis

Let Σ be the alphabet of a programming language, T a finite set of **tokens**, and A an arbitrary set of **attributes**.

A **tokenizer** is a function

$$\textit{tokenize} : \Sigma^* \rightarrow (T \times A)^*$$

such that there is a function

$$\textit{untokenize} : (T \times A)^* \rightarrow \Sigma^*$$

Lexical Analysis

... with the following properties:

1. $tokenize \circ untokenize = id_{(T \times A)^*}$ and
2. there is a function $untoken : (T \times A) \rightarrow \Sigma^*$ so that

$$untokenize(t_1 t_2 \dots) = untoken(t_1) untoken(t_2) \dots$$

($untokenize$ is a **homomorphism**).

Regular Expressions

$RE(\Sigma)$ is the smallest set with:

- $\underline{\emptyset} \in RE(\Sigma)$
- $\underline{\varepsilon} \in RE(\Sigma)$
- if $a \in \Sigma$ then $\underline{a} \in RE(\Sigma)$
- if $r_1, r_2 \in RE(\Sigma)$ then $r_1 r_2 \in RE(\Sigma)$
- if $r_1, r_2 \in RE(\Sigma)$ then $r_1 \mid r_2 \in RE(\Sigma)$
- if $r \in RE(\Sigma)$ then $r^* \in RE(\Sigma)$.

Language Defined by a Regular Expression

$$L : RE(\Sigma) \rightarrow \mathcal{P}(\Sigma^*)$$

$$L(\underline{\emptyset}) = \emptyset$$

$$L(\underline{\varepsilon}) = \{\varepsilon\}$$

$$L(\underline{a}) = \{a\}$$

Language Defined by a Regular Expression

$$L : RE(\Sigma) \rightarrow \mathcal{P}(\Sigma^*)$$

$$L(r_1 r_2) = L(r_1) \cdot L(r_2)$$

$$:= \{w_1 w_2 \mid w_1 \in L(r_1), w_2 \in L(r_2)\}$$

$$L(r_1 \mid r_2) = L(r_1) \cup L(r_2)$$

$$L(r^*) = L(r)^*$$

$$:= \{w_1 w_2 \dots w_n \mid n \in \mathbb{N}, w_i \in L(r)\}$$

$$= \{\varepsilon\} \cup L(r) \cup L(r) \cdot L(r) \cup L(r) \cdot L(r) \cdot L(r) \cup \dots$$

Lexer Implementation Strategies

- translate to NFA, DFA (Dragon book)
- derivatives (Matt Might: A regular expression matcher in Scheme using derivatives)
- issue: keywords may cause automaton explosion

Datalog Example

```
#lang datalog
% path test from Chen & Warren
edge(a, b). edge(b, c). edge(c, d). edge(d, a).

node(X) :- edge(X, Y).
node(Y) :- edge(X, Y).

edge(X, X) :- node(X).

path(X, X) :- node(X).
path(X, Y) :- edge(X, Y).
path(X, Y) :- edge(X, Z), path(Z, Y).
path(X, Y) :- path(X, Z), edge(Z, Y).

node(X)?
edge(X, Y)?
path(X, Y)?
```

Datalog Lexer in Racket

```
#lang racket/base
(require parser-tools/lex
         (prefix-in : parser-tools/lex-sre))

(define-tokens dtokens
  (VARIABLE IDENTIFIER STRING))
(define-empty-tokens dpunct
  (LPAREN COMMA RPAREN TSTILE DOT EQUAL NEQUAL TILDE QMARK EOF))
(define-lex-abbrev line-break #\newline)
(define-lex-abbrev id-chars
  (char-complement (char-set "(,)=:~?\"% \n")))
(define-lex-abbrev variable-re
  (:: upper-case
     (:* (:or upper-case lower-case (char-set "0123456789_")))))
(define-lex-abbrev identifier-re
  (:: id-chars (:* (:or upper-case id-chars))))
(define-lex-abbrev comment-re
  (:: "%"
     (complement (:: any-string line-break any-string)
                  line-break))
```

Datalog Lexer in Racket

```
(define get-string-token
  (lexer
    [(eof) (error 'datalog-lexer "Unterminated string")]
    [(:~ #\" #\\ #\newline)
     (cons (car (string->list lexeme))
           (get-string-token input-port))]
    [(:~ #\\ #\\)
     (cons #\\ (get-string-token input-port))]
    [(:~ #\\ #\newline)
     (cons #\newline (get-string-token input-port))]
    [(:~ #\\ #\")
     (cons #\" (get-string-token input-port))]
    [#\" null]))
```

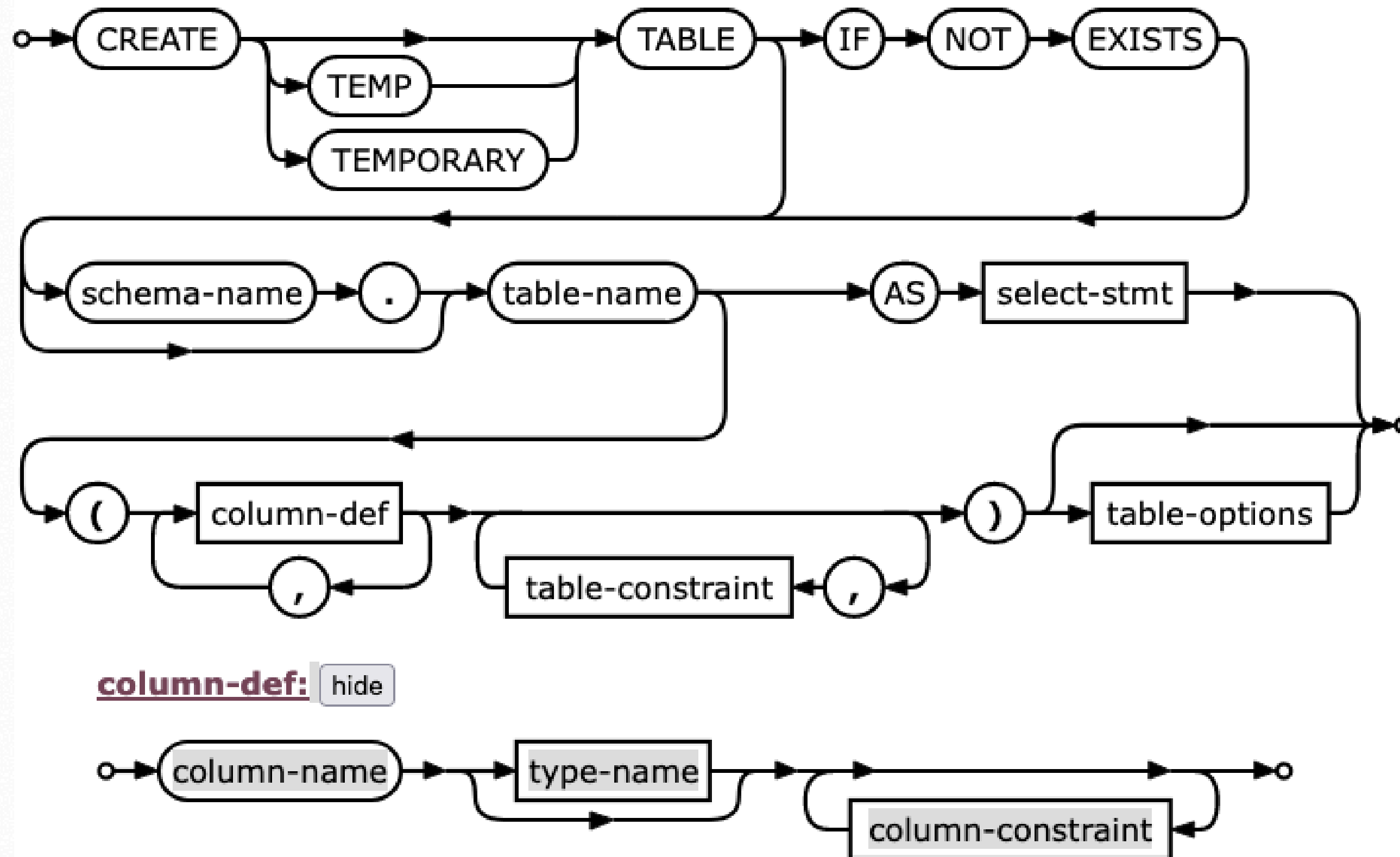
Datalog Lexer in Racket

```
(define dlexer
  (lexer-src-pos
    [whitespace
      (return-without-pos (dlexer input-port))]
    [comment-re
      (return-without-pos (dlexer input-port))]
    [variable-re
      (token-VARIABLE lexeme)]
    [identifier-re
      (token-IDENTIFIER lexeme)]
    [":-" (token-TSTILE)]
    [#\" (token-STRING (list->string (get-string-token input-port)))]
    [#\" (token-LPAREN)]
    [#\", (token-COMMA)]
    [#\" (token-RPAREN)]
    [#\" (token-DOT)]
    [#\"~ (token-TILDE)]
    [#\"? (token-QMARK)]
    [#\"= (token-EQUAL)]
    [\"!=\" (token-NEQUAL)]
    [(eof) (token-EOF)]))
```

(LG 2-2) Limits of Regular Expressions

- no matched parens / brackets / braces
- no recursive definitions

Syntactic Structure



SQLite Syntax Documentation

Context-Free Grammar

A **context-free grammar** is a tuple $G = (N, T, P, S)$. N is the set of **nonterminals**, T the set of **terminals**, $S \in N$ the **start symbol**, $V = T \cup N$ the set of **grammar symbols**. P is the set of **productions**; productions have the form $A \rightarrow \alpha$ for a nonterminal A and a sequence α of grammar symbols.

ϵ is the empty sequence; $|\xi|$ is the length of sequence ξ . Furthermore, α^k denotes a sequence with k copies of α , and $\xi|_k$ is the sequence consisting of the first k terminals in ξ .

(LG 2-2) Chomsky Hierarchy

Type	languages	productions
Type-3	regular	$A \rightarrow a, A \rightarrow aB, A \rightarrow Ba$
Type-2	context-free	$A \rightarrow \alpha$
Type-1	context-sensitive	$\alpha A \beta \rightarrow \alpha \gamma \beta$
Type-0	recursively enumerable	$\gamma \rightarrow \alpha$

Conventions

Some letters denote elements of certain sets by default:

$$A, B, C, E \in N$$

$$\xi, \rho, \tau \in T^*$$

$$x, y, z \in T$$

$$\alpha, \beta, \gamma, \delta, \nu, \mu \in V^*$$

$$X, Y, Z \in V$$

All grammar rules in the text are implicitly elements of P .

Context-Free Grammar Example

Expression $\rightarrow$ *Expression plus Expression*

Expression $\rightarrow$ *Expression minus Expression*

Expression $\rightarrow$ *Expression asterisk Expression*

Expression $\rightarrow$ *Expression slash Expression*

Expression $\rightarrow$ *Expression percent Expression*

Expression $\rightarrow$ *identifier*

Expression $\rightarrow$ *literal*

Expression $\rightarrow$ *lparen Expression rparen*

Context-Free Grammar Example

Additive → *Multiplicative*

Additive → *Additive plus Multiplicative*

Additive → *Additive minus Multiplicative*

Multiplicative → *Primary*

Multiplicative → *Multiplicative asterisk Primary*

Multiplicative → *Multiplicative slash Primary*

Multiplicative → *Multiplicative percent Primary*

Primary → *identifier*

Primary → *literal*

Primary → *lparen Additive rparen*

BNF Example

Additive ::= *Multiplicative*
| *Additive Addop Multiplicative*

Addop ::= *plus* | *minus*

Multiplicative ::= *Primary*
| *Multiplicative multop Primary*

Multop ::= *asterisk* | *slash* | *percent*

Primary ::= *identifier*
| *literal*
| *lparen Additive rparen*

EBNF Example

Additive ::= *Multiplicative* (*Addop Multiplicative*)*

Addop ::= *plus* | *minus*

Multiplicative ::= *Primary* (*Multop Primary*)*

Multop ::= *asterisk* | *slash* | *percent*

Primary ::= *identifier*

| *literal*

| *lparen Additive rparen*

Derives Relation

G induces the **derives relation** $\Rightarrow$ on V^* with

$$\alpha \Rightarrow \beta :\Leftrightarrow \alpha = \delta A \gamma \wedge \beta = \delta \mu \gamma \wedge A \rightarrow \mu$$

and $\Rightarrow^*$ denotes the reflexive and transitive closure of $\Rightarrow$.

A **derivation** from α_0 to α_n is a sequence $\alpha_0, \alpha_1, \dots, \alpha_n$ where $\alpha_{i-1} \Rightarrow \alpha_i$ for $1 \leq i \leq n$.

A **sentential form** is a sequence appearing in a derivation beginning with the start symbol.

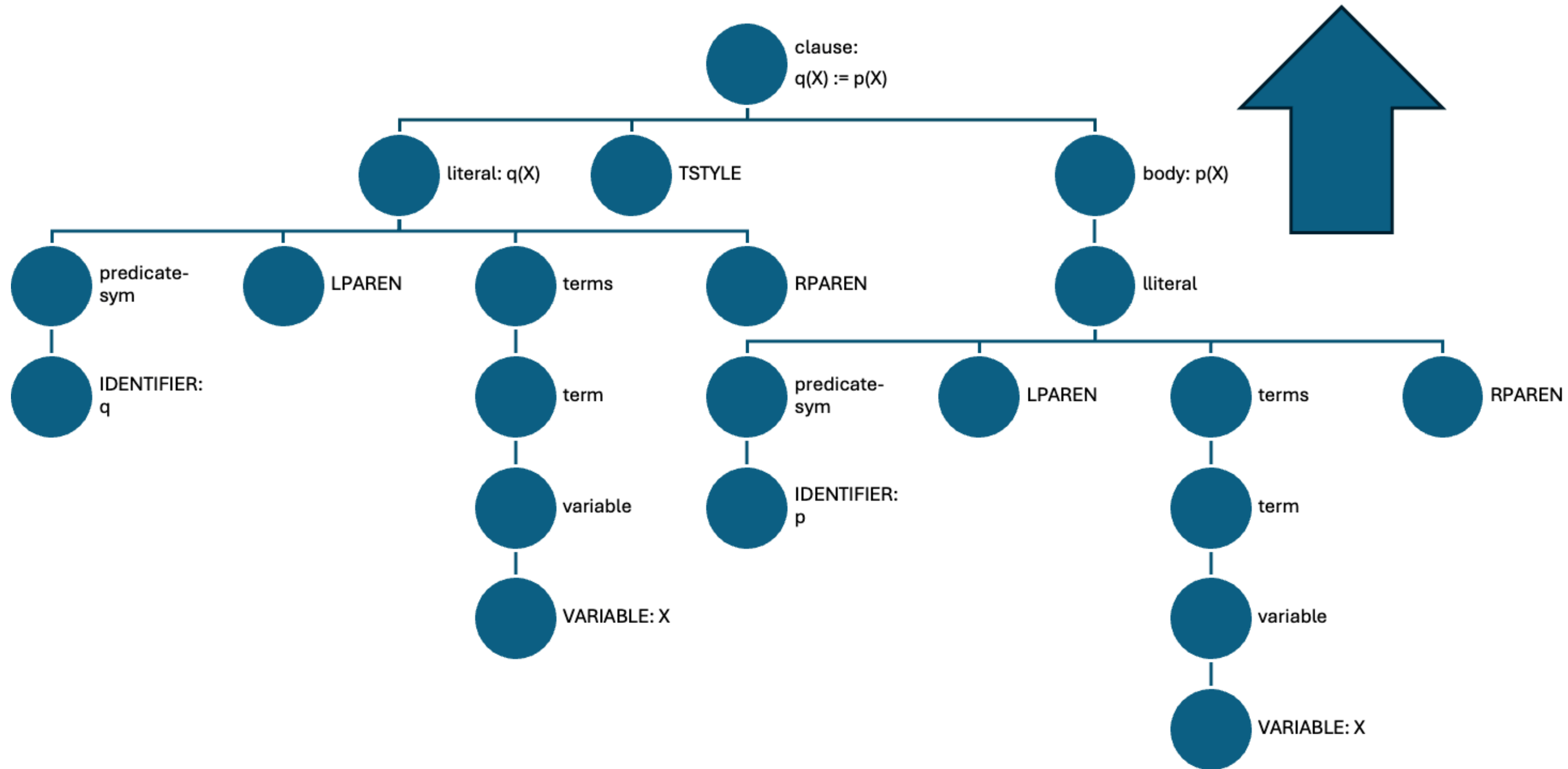
(LG 1-2) Parsing

$$[X] : T^* \rightarrow \mathcal{P}(T^*)$$

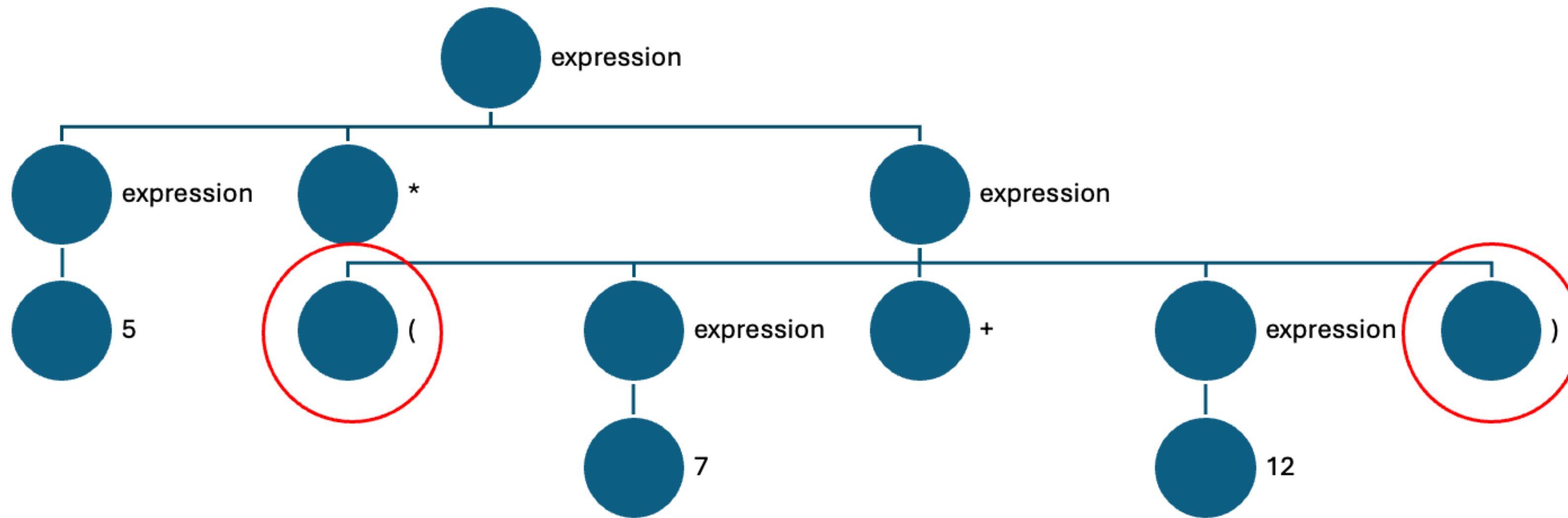
$$[X](x_1 \dots x_n) = \{x_{k+1} \dots x_n \mid X \xRightarrow{*} x_1 \dots x_k\}$$

ξ is in the language defined by the grammar iff $\epsilon \in [S](\xi)$.

Attribute Grammar



(LG 1-2, LG 2-3) Abstract Syntax



(LG 2-3) Datalog Abstract Syntax

```
(struct node (srcloc))
```

```
(struct assertion node (clause))
```

```
(struct retraction node (clause))
```

```
(struct query node (question))
```

```
(struct requirement node (lib))
```

```
...
```

Datalog Parser

```
(define-values
  (program-parser statement-parser clause-parser literal-parser)
  (apply
    values
    (parser
      (start program statement clause literal)
      (end EOF)
      (tokens dtokens dpunct)
      (src-pos) (error ...)
      (grammar
        (program [(statements) $1])
        (statements [() empty]
                    [(statement statements) (list* $1 $2)])
        (statement [(assertion) $1]
                  [(query) $1]
                  [(retraction) $1]
                  [(requirement) $1])
        ...
        (requirement [(LPAREN IDENTIFIER RPAREN DOT)
                      (make-requirement (make-srcloc $1-start-pos $4-e

```

(LG 2-3) Abstract Syntax

Let Σ be the alphabet of the language of a program. **Abstract syntax generation** is a function

$$\textit{parse} : \Sigma^* \rightarrow D$$

where D is a suitable set such that a function

$$\textit{unparse} : D \rightarrow \Sigma^*$$

exists with

$$\textit{parse} \circ \textit{unparse} = \textit{id}_D.$$

(LG 2-3) Exercise: Abstract Syntax for Tim

Design an abstract syntax for Tim and implement it in Racket!

"Parser"

$$\begin{aligned}[\epsilon](\xi) &= \{\xi\} \\[a](a\xi') &= \{\xi'\} \\[a](b\xi') &= \emptyset \text{ if } a \neq b \\[X\alpha](\xi) &= \bigcup \{[\alpha](\rho) \mid \rho \in [X](\xi)\} \\[A](\xi) &= \bigcup_{A \rightarrow \alpha} [\alpha](\xi)\end{aligned}$$

- non-deterministic
- doesn't work for left recursion

Lookahead

$$\text{first}_k : V^* \rightarrow T^k$$

$$\text{first}_k(\alpha) := \{\xi_{|k} \mid \alpha \xRightarrow{*} \xi\}$$

$$\text{follow}_k : N \rightarrow T^k$$

$$\text{follow}_k(A) := \{\xi \mid S \xRightarrow{*} \beta A \gamma \wedge \xi \in \text{first}_k(\gamma)\}$$

Recursive-Descent/LL Parser

$$[A](\xi) = \bigcup_{A \rightarrow \alpha, \xi|_k \in (\text{first}_k(\alpha) \text{ follow}_k(A))|_k} [\alpha](\xi)$$

AKA top-down parser

(LG 2-2) LL(k) Grammars

The **LL(k) lookahead** of a production $A \rightarrow \alpha$ is computed as follows:

$$LLA_k(A \rightarrow \alpha) := (\text{first}_k(\alpha) \text{ follow}_k(A))|_k$$

A context-free grammar G is **LL(k)** if, for productions $A \rightarrow \alpha$ and $A \rightarrow \beta$ with $\alpha \neq \beta$,

$$LLA_k(A \rightarrow \alpha) \cap LLA_k(A \rightarrow \beta) = \emptyset.$$

Caveats of LL/Recursive Descent Parsing

- decision on production is made on the **left**
- no left recursion

BNF Example, Suitable for LL Parsing

Additive ::= Multiplicative MoreAdditive

MoreAdditive ::= ϵ | Addop MoreAdditive

Addop ::= plus | minus

Multiplicative ::= Primary MoreMultiplicative

MoreMultiplicative ::= ϵ | multop MoreMultiplicative

Multop ::= asterisk | slash | percent

*Primary ::= identifier | literal
| lparen Additive rparen*

(LG 2-1) Exercise: Tim Grammar und Parser

Design a grammar for the Tim language, along with a reursive-descent parser for it in Racket.

(LG 2-2) LR Parsing

- also works from left to right
- ... but only decides on a production at its **right** edge, i.e. *later*
- ... and therefore works on a larger class of grammars

LR Parsing

An **LR(k) item** (or just **item**) is a triple consisting of a production, a position within its right-hand side, and a terminal string of length k - -the **lookahead**.

An item is written as $A \rightarrow \alpha \cdot \beta (\rho)$ where the dot indicates the position, and ρ is the lookahead.

If the lookahead is not used (or $k = 0$), it is omitted.

A **kernel item** has the form $A \rightarrow \alpha \cdot \beta (\rho)$ with $|\alpha| > 0$.

A **predict item** has the form $A \rightarrow \cdot \alpha (\rho)$.

An **LR(k) state** (or just **state**) is a non-empty set of LR(k) items.

LR Parsing

Each state q has an associated set of **predict items**:

$$\text{predict}(q) := \left\{ B \rightarrow \cdot \nu (\tau) \mid \begin{array}{l} A \rightarrow \alpha \cdot \beta (\rho) \Downarrow^+ B \rightarrow \cdot \nu (\tau) \\ \text{for } A \rightarrow \alpha \cdot \beta (\rho) \in q \end{array} \right\}$$

where $\Downarrow^+$ is the transitive closure of the relation $\Downarrow$ defined by

$$A \rightarrow \alpha \cdot B \beta (\rho) \Downarrow B \rightarrow \cdot \delta (\tau) \text{ for all } \tau \in \text{first}_k(\beta \rho)$$

LR Parsing

The union of q and $\text{predict}(q)$ is called the **closure** of q :

$$\bar{q} := q \cup \text{predict}(q)$$

denotes the closure of a state q .

For a state q and a grammar symbol X :

$$\text{goto}(q, X) := \{A \rightarrow \alpha X \cdot \beta (\rho) \mid A \rightarrow \alpha \cdot X \beta (\rho) \in \bar{q}\}$$

$$\text{nextterm}(q) := \{x \mid A \rightarrow \alpha \cdot x \beta \in \bar{q}\}$$

$$\text{nactive}(q) := \max\{|\alpha| : A \rightarrow \alpha \cdot \beta \in q\}$$

Recursive-Ascent Parsing

$[q](\xi, c_1, \dots, c_{\text{nactive}(q)}) :=$
letrec $c_0(X, \xi) = [\text{goto}(q, X)](\xi, c_0, c_1, \dots, c_{\text{nactive}(\text{goto}(q, X)) - 1})$
in
 $A \rightarrow \alpha \cdot (\rho) \in \bar{q} \wedge \xi|_k = \rho \triangleright c_{|\alpha|}(A, \xi)$ (reduce)
 $\xi = x\xi' \wedge x \in \text{nextterm}(q) \triangleright c_0(x, \xi')$ (shift)

Tools

- Racket: LR
- yacc: LR
- Bison: LR
- ANTLR: LL
- Xtext: LL (ANTLR)
- Textx: LL
- Lezer: LR
- Langium: LL
- Spoofax: LR

(LG 5-1) Exercise: Tim Grammar und Parser

Rewrite the grammar using `parser-tools/yacc`, and generate abstract syntax.

(L 2-3) Exercise: JSON/YAML/XML-based Syntax

You could design a syntax based on JSON, YAML or XML. Find examples for this approach. Discuss advantages and disadvantages compared to a syntax designed and implemented from scratch.

(LG 2-3, LG 5-2, LG 5-4) Exercise: Xtext DSL

Implement the grammar for the previous exercise in Xtext, and evolve it into a DSL implementation.

(LG 2-3, LG 5-4) Exercise: MPS DSL

Design a projectional representation for the syntax from the previous exercise, and evolve it into a DSL implementation.